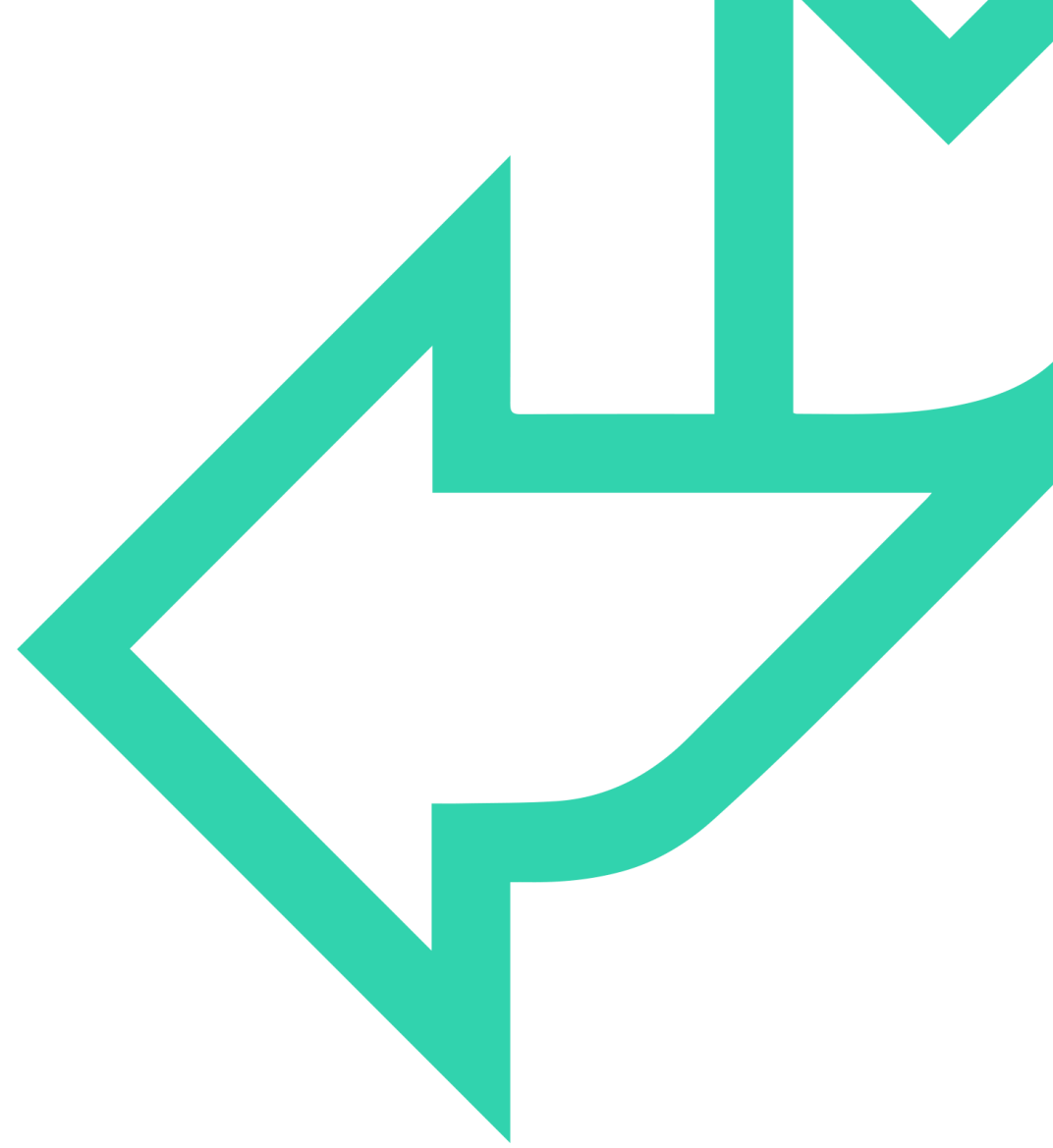




Classes and OOP





CLASSES

- Classes – a set of attributes and methods
 - Attribute – data
 - Method - code or function
- Object - an instance of a class

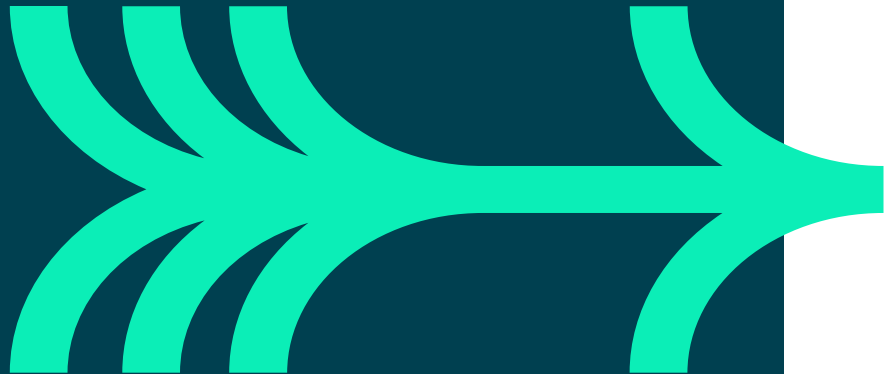




CLASS CONSTRUCTOR

Class: Dog
Attributes:
Breed
Weight
Energy

Object: Bilbo Waggins
Breed: Labrador
Weight: 80lbs
Energy: Low





CLASSES IN PYTHON



```
class Dog:
    energy = "high"
    def speak(self):
        print("woof")
```

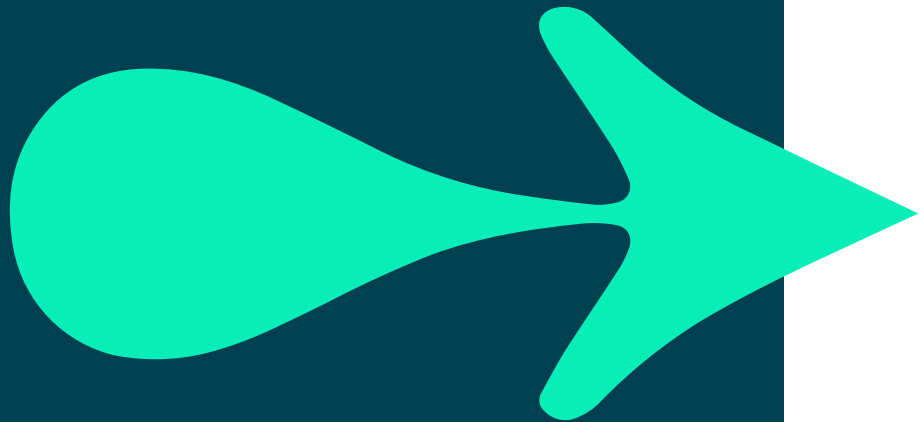
```
bilbo_waggins = Dog()
print(bilbo_waggins.energy)
bilbo_waggins.speak()
```



CLASS CONSTRUCTOR

```
class Dog:
    def __init__(self, name, breed, energy):
        self.name = name
        self.breed = breed
        self.energy = energy

dog1 = Dog("Ross Barkley", "Jack Russel", "High")
```





SELF

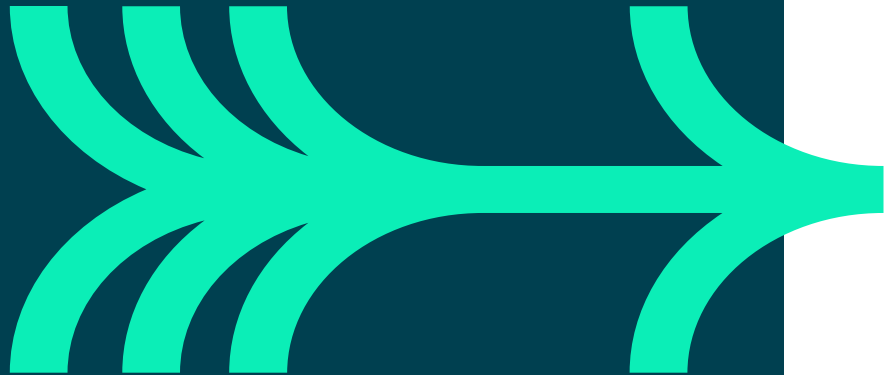
- The self parameter is a reference to the current instance of the class
- The parameter does not need to be called self, that's just best practice, but it must be the first parameter of any class function!





CLASS METHODS

- `getattr(object, name)` - Accesses the named attribute of a specific object
- `hasattr(object, name)` - Checks if a specific object has a named attribute
- `setattr(object, name, value)` - Sets the named attribute of a specific object as the value specified
- `delattr(object, name)` - Deletes the named attribute of a specific object





UNDERScores

- Single leading underscore - `_money = 10000`
- Double leading underscore - name mangling
- Single trailing underscore - Python keywords





OOP

- Inheritance
- Polymorphism
- Encapsulation
- Abstraction





INHERITANCE



```
class Athlete:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class Footballer(Athlete):
```

```
    def __init__(self, name, team):  
        super(Footballer, self).__init__(name)  
        self.team = team
```



POLYMORPHISM

- Overloading

```
class Bird:
    def __init__(self, wingspan):
        self.wingspan = wingspan

    def __len__(self):
        return self.wingspan
```

```
eagle = Bird(104)
```

- Overriding

```
class Animal:
    babies = 0

    def reproduce(self):
        self.babies += 1
```

```
class dog(Animal):
    def reproduce(self):
        self.babies += 6
```



ENCAPSULATION



```
class BankAccount:
```

```
    _money = 0
```

```
    def deposit(self, money, password):
```

```
        if (authenticate(password)):
```

```
            self.money += money
```

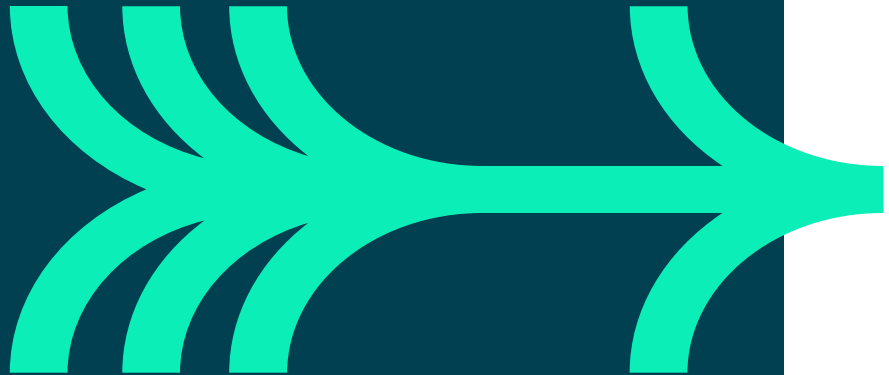
```
    def withdraw(self, money, password):
```

```
        if (authenticate(password)):
```

```
            self.money -= money
```



ABSTRACTION



```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):  
    @abstractmethod  
    def eat(self):  
        pass
```

```
class Mammal(Animal):  
    def eat(self):  
        print("nom nom")
```



Guided Task

→ Please open the instructions in Cloud Academy



Thank you!

Hope you enjoyed this learning journey!